Desenvolvimento de um sistema de comércio eletrônico para a Loja Giacomoni

Dimitri Valssoler Krewer
Curso de Análise e Desenvolvimento de Sistemas
Universidade Luterana do Brasil campus Torres
Orientador Daniel Ricardo de Souza
2026

Resumo

Este trabalho apresenta o desenvolvimento de um sistema de comércio eletrônico para a Loja Giacomoni, voltada à venda de pijamas e moda íntima. O projeto atende o cliente com catálogo, busca, seleção de variações, carrinho e checkout. Também oferece uma área administrativa protegida para cadastro de produtos, controle de estoque e acompanhamento dos pedidos. A solução usa React no frontend, Node.js com Express na API e MongoDB para persistência. Os testes locais confirmaram a criação de pedidos, a baixa condicional do estoque e a devolução automática das unidades após cancelamento. O resultado é um produto mínimo viável executável, documentado e preparado para publicação em serviços de nuvem.

Palavras chave: comércio eletrônico, React, Node.js, MongoDB, controle de estoque.

Introdução

Pequenos comércios dependem cada vez mais de canais digitais para apresentar produtos e organizar vendas. A Loja Giacomoni precisava de uma solução que permitisse ao cliente consultar peças e realizar um pedido sem depender de atendimento manual em todas as etapas. Ao mesmo tempo, a administração precisava manter preços, variações e estoque em um único lugar.

O objetivo geral do projeto foi desenvolver um sistema web de comércio eletrônico para a loja. Os objetivos específicos foram criar uma experiência de compra responsiva, implementar uma API com regras de negócio, persistir produtos e pedidos, proteger a administração e validar os fluxos mais importantes por meio de testes.

Fundamentação tecnológica

O React organiza a interface em componentes e atualiza a tela conforme o estado da aplicação. O Vite fornece o ambiente de desenvolvimento e gera arquivos otimizados para produção. Essa combinação permitiu separar a vitrine, a página de produto, o carrinho, o checkout e a administração.

A API usa Node.js e Express. O Express define rotas HTTP para produtos, autenticação, pedidos e indicadores administrativos. O Mongoose representa os dados do MongoDB e aplica validações antes da gravação. A autenticação usa JSON Web Token, enquanto o bcrypt armazena somente o hash da senha administrativa.

O MongoDB foi escolhido porque produtos e pedidos possuem estruturas naturalmente documentais. Cada produto contém suas variações de tamanho, cor e quantidade. Cada pedido preserva os itens, os valores usados na compra, o endereço e o histórico de status.

Metodologia de desenvolvimento

O desenvolvimento seguiu entregas incrementais. A primeira etapa criou a identidade visual e a navegação da loja. Em seguida foram implementados catálogo, pesquisa, filtros, detalhes do produto e carrinho. A terceira etapa acrescentou checkout, persistência e regras de estoque. Por fim, o painel administrativo recebeu autenticação, manutenção de produtos, indicadores e gestão de pedidos.

A validação combinou testes automatizados das regras puras com um roteiro integrado executado contra a API e um banco MongoDB local. O frontend também foi compilado em modo de produção para detectar erros de integração e empacotamento.

Arquitetura do sistema

O navegador executa o frontend React e envia requisições à API. A API centraliza validações e regras de negócio antes de acessar o MongoDB. Essa divisão evita que decisões de estoque e autorização dependam apenas da interface do usuário.

As rotas públicas permitem listar produtos e criar pedidos. As rotas protegidas exigem um token administrativo para alterar o catálogo, consultar pedidos, atualizar status ou acessar indicadores. Em produção, a API limita as origens aceitas por meio da configuração de CORS.

Funcionalidades implementadas

A loja apresenta produtos ativos, oferece pesquisa e filtros, mostra detalhes e exige a escolha de tamanho e cor. O carrinho guarda os itens no navegador, permite alterar quantidades e calcula o total. O checkout coleta os dados do cliente e o endereço de entrega antes de enviar o pedido.

O painel administrativo permite entrar com credenciais seguras, acompanhar produtos e pedidos, cadastrar ou editar itens e controlar o estoque de cada variação. A listagem de pedidos aceita filtros por cliente, data e status. A tela de detalhes exibe endereço, itens, valores e histórico.

Ao criar um pedido, a API reduz o estoque somente quando há quantidade suficiente. Se uma etapa falhar, as unidades já reservadas retornam ao produto. O cancelamento também devolve o estoque, com uma marcação que impede devoluções repetidas.

Testes e resultados

Os cinco testes automatizados foram aprovados. Eles cobrem o agrupamento do mesmo produto e variação, a separação de variações distintas, a rejeição de itens inválidos, as transições permitidas de status e o tratamento seguro dos caracteres informados na pesquisa.

O teste integrado usou dois produtos cadastrados no banco local. O login administrativo foi aceito, um pedido foi criado e o estoque escolhido caiu de quatro para três unidades. Após o cancelamento, a quantidade voltou para quatro. A compilação do frontend também terminou sem erros.

Esses resultados comprovam os fluxos centrais do produto mínimo viável. O pagamento ocorre fora da plataforma, portanto o sistema registra o pedido, mas não processa transações financeiras.

Publicação e manutenção

O projeto inclui contêineres para frontend e backend, um ambiente MongoDB local e arquivos de configuração para Render e Vercel. A integração contínua executa testes e compilação a cada alteração enviada ao repositório. A publicação final exige as contas do proprietário e a definição de credenciais exclusivas de produção.

As variáveis sensíveis ficam fora do código. Em produção, o responsável deve usar uma chave JWT longa, uma senha administrativa nova e uma conexão restrita do MongoDB Atlas. Após a publicação, os fluxos de catálogo, checkout e administração devem ser repetidos no endereço público.

Conclusão

O projeto alcançou o objetivo de implementar um comércio eletrônico funcional para a Loja Giacomoni. A solução integra a experiência do cliente ao controle administrativo e mantém a regra de estoque no servidor. Os testes confirmaram os principais comportamentos e a estrutura de publicação permite transferir o sistema para serviços de nuvem.

Como continuidade, o projeto pode incorporar pagamento online, cálculo de frete, recuperação de senha, envio de notificações e testes de interface automatizados. Essas extensões não impedem o uso acadêmico e demonstrativo da versão atual.

Tecnologias e responsabilidades

Tecnologia	Uso no projeto	Motivo da escolha
React e Vite	Interface da loja e do painel	Componentização e compilação rápida
Node.js e Express	API e regras de negócio	Mesma linguagem entre cliente e servidor
MongoDB e Mongoose	Persistência e validação	Adequação a produtos com variações
JWT e bcrypt	Sessão e senha administrativa	Proteção das rotas e do segredo
Docker	Ambiente local e empacotamento	Execução reproduzível

Referências

FACEBOOK. React documentation. Disponível em https://react.dev. Acesso em 8 nov. 2026.

OPENJS FOUNDATION. Node.js documentation. Disponível em https://nodejs.org/docs. Acesso em 8 nov. 2026.

MONGODB. MongoDB documentation. Disponível em https://www.mongodb.com/docs. Acesso em 8 nov. 2026.

EXPRESS. Express documentation. Disponível em https://expressjs.com. Acesso em 8 nov. 2026.